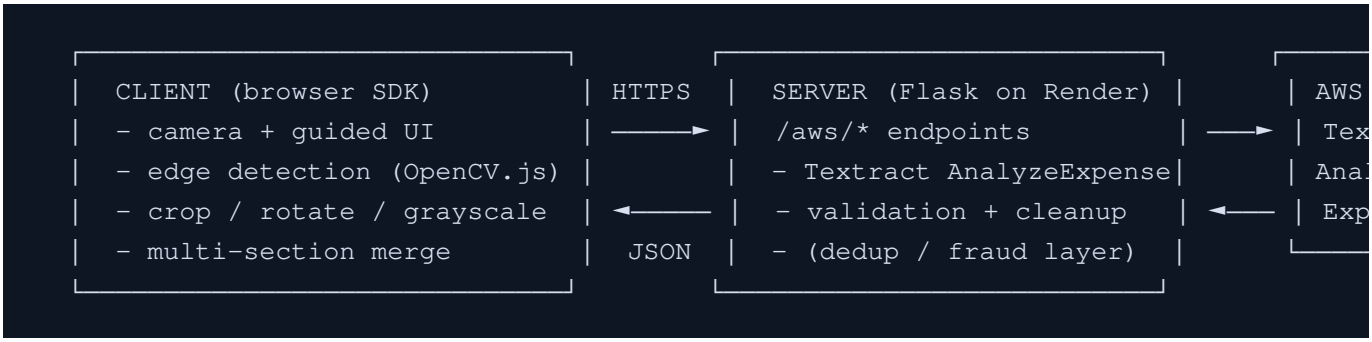
Receipt Capture Web SDK — Technical Specification

`ReceiptCaptureSDK` is a dependency-free, browser-based SDK that captures a receipt with the device camera (or an uploaded image), prepares an OCR-optimized payload, and delivers it to a cloud backend that runs **Amazon Textract**. It ships as a single file — `static/receipt-capture-sdk.js` — and is consumed by the member UI in `design_preview_noclip.html`.

---

1. Architecture — Cloud-Hybrid

Work is split between the device and the cloud for performance and security.



Security boundary. No fraud validation, OCR, or database logic runs on the client. The SDK only *prepares and delivers* media. All extraction, validation, and (when enabled) deduplication happen server-side. The browser never holds AWS credentials — it talks only to the app's own `/aws/*` endpoints.

Layer	Responsibilities
Client (SDK)	Camera control, guidance UI, live edge detection, local image quality checks (luminance), DPR-aware cropping, orientation normalization, grayscale JPEG compression, multi-section capture + merge, upload + polling
Server (AWS)	Textract AnalyzeExpense extraction, readability/confidence gating, temporal rules, line-item cleanup, optional cryptographic + semantic deduplication

---

2. What the SDK does

2.1 Camera capture

- Requests the **rear camera** with `facingMode: { exact: "environment" }` and `advanced: [{ focusMode: "continuous" }]` for close-up text focus.
- **Graceful fallback** to `facingMode: "environment"` (then default sensor) if the exact constraint

fails.

- Ideal capture resolution **1920×1080**.
- The injected `<video>` is forced `playsinline`, `autoplay`, `muted` (both as attributes and properties) for inline rendering on iOS Safari / mobile.

2.2 Guided full-screen UI

A full-viewport overlay with two modes:

LIVE mode

- A **fixed bounding box** (portrait): a top edge + two full-height side rails, open at the bottom so long receipts can extend past it. The box never moves.
- **Per-edge detection:** each edge turns **green** ✓ when the receipt's matching edge lands within tolerance of that fixed box edge, **amber ?** otherwise. This enforces that the receipt fills the frame at a size Textract can read. The **bottom edge** appears only when the receipt's end is actually in view.
- **Manual capture only** — the user taps the shutter; the SDK never auto-fires.
- **Lighting coaching:** "Too dark", "Too bright", "Glare — tilt the receipt".
- **Flash/torch toggle** (shown only if the camera track exposes `torch`).
- **"Photo N" counter**, help panel, and a long-receipt hint.
- For continuation sections, a tinted **overlap sliver** of the previous section's bottom is pinned to the top to help the user line up the next part.

REVIEW mode (after each capture)

- The frozen still + a **"Store** / **Date/Time** / **Total"** field checklist.
- Actions: **Retake** · **Use receipt photo** · **Long receipt? Add section**.

2.3 Edge detection (OpenCV.js)

- **Lazy-loaded** (~8 MB WASM) from a CDN when the camera opens; **graceful fallback** to the fixed guide box if it fails to load.
- Detection is **brightness-band based** (the receipt is lighter than its background) rather than gradient-based — this is robust to interior text, which gradient methods mistake for an edge.
- **Two gates prevent false positives** (edges lighting up with no receipt):
    1. **Contrast gate** — the bright band must clearly exceed the background.
    2. **Text-density gate** — Canny edge density inside the band must be high enough to be *text*. A blank wall, table, or hand is rejected.
- An edge is reported "in view" only when the bright band **starts/ends inside the frame**; touching a border means that edge is off-screen.

2.4 Image processing & normalization

- **DPR-aware crop:** the fixed box region is mapped onto the raw sensor matrix via `getBoundingClientRect()` ratios and rasterized at `window.devicePixelRatio` for crisp output on Retina/OLED displays.

- **Skew correction:** if the sensor outputs landscape but the layout is portrait, the canvas is rotated **90° clockwise** before extraction.
- **Luminance guardrail:** mean Rec.709 relative luminance must be within **[40, 230]** (else the capture is rejected with a coach message).
- **Compression:** output is a **grayscale JPEG at quality 0.85**.

2.5 Multi-section long receipts

- The user captures the receipt in sections (top → "Add section" → next …).
- Sections are **not pixel-stitched** (fragile on thermal paper). Instead each section is sent to **Textract independently** and the structured results are **merged**.
- **Overlap removal:** the capture overlap is a contiguous run of lines shared between sections. The merge finds where one section's leading run matches a contiguous run **anywhere in the accumulated items** (by description, since OCR totals drift) and drops it **once**.
- **Repeats preserved:** because it matches *runs* (not individual items), legitimately repeated purchases (e.g. 3× the same product) are kept.

2.6 Secure transmission + polling

For each section (or the single image):

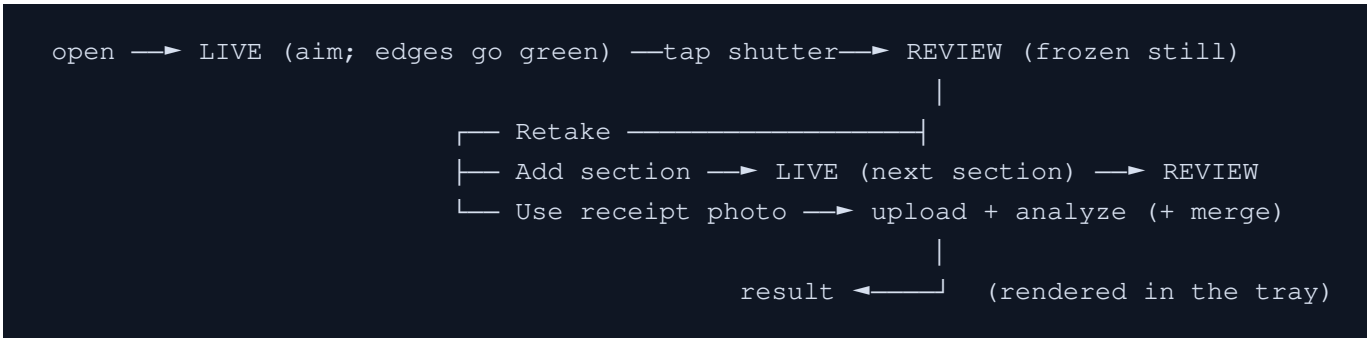
1. `POST /aws/get-upload-url` → returns a `job_id` and a time-limited PUT URL (a local stand-in for an S3 presigned PUT).
2. `PUT` the compressed JPEG bytes to that URL.
3. Poll `GET /aws/analyze-status?job=<id>` every **2 s** until terminal (`done` / `rejected` / `failed`), avoiding gateway timeouts during OCR.

2.7 Device gating (host page)

- **Desktop → upload only.** The camera button is hidden; the tray offers "Upload a receipt".
- **Mobile / tablet → both.** Upload *or* "Capture receipt with camera".
- Detection uses `pointer: coarse` (+ user-agent) so iPads count as tablets while mouse-driven touch laptops count as desktop.
- The upload path runs through the **same AWS pipeline** (get-url → PUT → poll).

---

3. Capture flow



4. Public API

Constructor

```
const sdk = new ReceiptCaptureSDK(options);
const result = await sdk.captureFlow();   // opens the full-screen UI
```

Options

Option	Default	Description
`endpoints.uploadUrl`	`/aws/get-upload-url`	Endpoint that returns a job id + PUT URL
`endpoints.status`	`/aws/analyze-status`	Poll endpoint
`userId`	`"demo-user"`	Sent as `X-User-Id` on upload (used for dedup)
`idealWidth` / `idealHeight`	`1920` / `1080`	Requested capture resolution
`jpegQuality`	`0.85`	Output JPEG quality
`grayscale`	`true`	Convert output to grayscale
`lumaMin` / `lumaMax`	`40` / `230`	Luminance guardrail bounds
`glareMax`	`0.14`	Max fraction of near-white pixels before a glare warning
`edgeDetection`	`true`	Enable OpenCV edge detection
`box`	`{left:0.08, right:0.92, top:0.11}`	Fixed bounding box (viewport fractions)
`alignTol`	`0.12`	±tolerance for an edge to count as aligned (green)
`contrastMin`	`26`	Min brightness range for a band to exist
`textureMin`	`0.035`	Min Canny edge-density (text) to be a real receipt
`pollIntervalMs`	`2000`	Status poll interval
`maxPolls`	`60`	Poll attempts before timeout (~2 min)

`opencvUrl`	`https://docs.opencv.org/4.8.0/opencv.js`	OpenCV.js source

Methods

Method	Description
`captureFlow()` → `Promise<result>`	Open the full-screen UI; resolves with the OCR result. Rejects with `{cancelled:true}` if the user closes it.
`submit()` → `Promise<result>`	Encode → upload → poll (per section, then merge). Used internally by the flow; callable directly with `segments` set.
`on(event, cb)`	Subscribe to events.
`close()` / `stop()`	Tear down the UI / stop the camera.
`segmentCount` / `clearSegments()` / `removeLastSegment()`	Manage captured sections.

Events

Event	Payload
`ready`	`{ width }` — camera started
`status`	`{ phase, message, attempt?, status?, bytes? }` — pipeline progress (`requesting-url`, `uploading`, `polling`, `detector-ready`, `fallback`)
`segment`	`{ count, luminance, ... }` — a section was captured

---

5. Backend API (`/aws/*`)

Implemented in `app.py` with Flask + boto3.

Endpoint	Purpose
`GET /aws/status`	Reports whether boto3 + AWS credentials are available
`POST /aws/get-upload-url`	Returns `{ job_id, key, upload_url }` (absolute, CORS-friendly)
`PUT /aws/upload/<job_id>`	Receives image bytes; starts Textract analysis in a background thread
`GET /aws/analyze-status?job=<id>`	`{ status: processing | done | rejected | failed, result?, error? }`

Normalized result shape

Textract `AnalyzeExpense` output is mapped to a stable shape the UI consumes:

```
{
  "vendor": { "name": "WHOLE FOODS MARKET" },
  "date": "06/13/2026",
  "total": 32.54,
  "currency_code": "USD",
  "document_type": "receipt",
  "line_items": [ { "description": "...", "total": 2.25, "quantity": "0.42 lb" } ],
  "line_item_count": 11,
  "avg_confidence": 95.38,
  "source": "aws-textract",
  "merged_sections": 2
}
```

Server-side validation layer

- **Readability hard block** — rejects if Textract returns no structural text.
- **Confidence gate** — rejects if average field confidence < `TEXTRACT_MIN_CONFIDENCE` (default 50).
- **Temporal rules** — rejects future-dated receipts or those older than `RECEIPT_MAX_AGE_DAYS` (default 14).
- **Line-item cleanup** — strips Textract's phantom rows (multi-line fragments, summary/payment lines like Subtotal/Tax/Total). Does **not** dedup by description, so legitimately repeated items are preserved.
- **Deduplication (anti-fraud)** — SHA-256 image-collision + semantic composite (`user_vendor_total_date`) checks. **Currently commented out / gated by `DEDUP_ENABLED`** during prototyping; re-enable for production.

Configuration (env)

`AWS_REGION` · `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` (or role) · `ALLOWED_ORIGIN` (CORS) · `RECEIPT_MAX_AGE_DAYS` · `TEXTRACT_MIN_CONFIDENCE` · `DEDUP_ENABLED`.

---

6. Tech stack & dependencies

Area	Technology
SDK	Vanilla ES (no build step), Canvas 2D, `getUserMedia`, MediaStream `torch`
Edge detection	OpenCV.js 4.8 (WASM, lazy-loaded from CDN)
Backend	Python · Flask · flask-cors · boto3 · gunicorn
OCR	Amazon Textract `AnalyzeExpense` (synchronous, image bytes)
Hosting	GitHub Pages (static frontend) + Render (Flask backend)

7. Limitations & production extension points

- **Edge/bottom detection** relies on the receipt being lighter than its background — works best on a **dark, non-shiny surface**; struggles on white tables. A heavier on-device ML model would improve this.
- **Overlap merge matches by description**; if the same line is OCR'd differently between sections, a duplicate can slip through. Adding fuzzy (edit-distance) matching would absorb that drift.
- **OpenCV.js is ~8 MB**, lazy-loaded on first camera open (one-time per session); on slow networks edge detection activates a few seconds in.
- **Backend is a lightweight stand-in:** Textract runs synchronously on the uploaded bytes (no S3), jobs/dedup live in process memory. Production would use **API Gateway + Lambda + S3 presigned PUT + async Textract + DynamoDB** for the dedup indexes — the client pipeline (presigned PUT + polling) already matches that shape, so only the server changes.
- **Textract `AnalyzeExpense` synchronous** path supports JPEG/PNG (not PDF); PDF/async would require the S3-backed flow.